

FreightLens Design Exercise

Data Platform for an Agentic Freight Analytics Product

Effort	Window	Deliverable	Next step
4-6 hours	4 days	Max 8 pages + diagrams	60-min discussion where you walk us through your design

Context

FreightLens is a B2B SaaS platform serving around 400 enterprise clients in global freight. The product is moving from dashboards to an AI-native experience. Clients now interact with agents that answer questions ("which shipments are at risk this week?"), generate insights ("your team booked spot rates on 60 lanes where contract rates were cheaper - estimated \$2.1M overspend"), and trigger actions like rebids, alerts, and tickets.

You are designing the data layer that these agents and systems sit on.

How data enters the platform

Path	What it brings
1. Core product systems	Bookings, contracts, tracking events, arriving via CDC into an OLAP store (assume ClickHouse).

Path	What it brings
2. AI extraction agents	For example, an email listener that parses forwarder emails into structured bookings. Extraction accuracy varies with document quality.
3. External sources	Port master data, freight rate indices, news feeds.

What the stakeholders are asking for

All of these are real asks, recorded in your discovery interviews:

Stakeholder	The ask
Product	"Agents should be able to answer any data question a client asks. The simplest thing is to let the agent write SQL directly, the models are good enough now."
Sales	"Our biggest prospect will only sign if they get a fully dedicated database instance. They've said isolation is their number one concern."
Data Science	"We want cross-client lane benchmarks - 'you pay 15% above market on China-EU'

Stakeholder	The ask
	- it's our strongest upsell hook."
Operations	"Last month an agent recommended a rebid based on a tracking event that was 3 days stale. The client escalated. This can't happen again."
Engineering	"The email extraction agent is wrong maybe 5-8% of the time. Those bookings look identical to real ones in the database."

Specific deliverables

Provide a unified document (maximum 8 pages plus technical diagrams) structured around the five key pillars detailed below. Maintain these headings for easier navigation.

1. Data organization design (~2 pages)

- Visual architectural flow tracing data from all three ingestion channels to the final consumption layer for agents and analytics. Ensure every stage is clearly labeled.
- Functional breakdown per stage: define inputs, specific transformations or validation checks, outputs, and handling of filtered or rejected records.
- Final schema sketches representing the primary tables or views queried by agents, highlighting core columns and their strategic role (no full DDL required).
- A concise justification defending why this specific architecture was chosen over simpler alternatives.

2. Access architecture (~2 pages)

- Mapping of all system participants—users, agents, pipelines, and extraction tools—detailing their specific access paths and enforced security constraints.
- Tenant isolation strategy: identify where scoping is enforced, the specific mechanism used, and the fallback behavior if an upstream component fails.
- Coexistence model explaining how dedicated database instances for high-tier prospects integrate with our standard multi-tenant infrastructure.
- The "intent-to-query" pipeline: describe how natural language is translated to SQL, including validation layers and rejection logic.

3. End-to-end trace (~1.5 pages)

Execute a full walkthrough of this scenario: "An agent assists a Client X operations user in identifying shipments with delayed ETDs this week and evaluating them for potential rebidding."

- A step-by-step sequential breakdown from the initial request to the final output.
- Component-level details for each step, specifying responsibilities, visibility constraints, and data access permissions.
- Analysis of how tracking data freshness is surfaced and its impact on the automated rebid logic.

4. Failure analysis (~1.5 pages)

Identify the two most significant failure modes within your design that could compromise enterprise operations. For each:

- Root cause analysis of the "silent failure"—why the system continues without triggering standard alerts.
- The resulting impact radius, specifically naming affected data marts, insights, or automated actions.
- Mitigation and detection: explain the design features that limit damage and how such issues are eventually identified.
- *Note: Focus on logical or data-level failures; assume base infrastructure (like ClickHouse) remains operational.*

5. Tradeoffs (~1 page)

- Discuss three intentional design omissions and the resulting functional costs of those decisions.
- Scalability limits: identify the first bottleneck encountered at 3x current volume and the required architectural shift.
- A critical self-assessment of your most controversial design choice and the data points that would lead you to reconsider it.

Constraints and notes

- Assume a small platform team of 4-5 engineers.
- Prefer boring technology. Name specific tools only where the choice is load-bearing. We are evaluating your reasoning, not tool trivia.
- You may use AI assistants while working on this. You will defend every decision live in the discussion round, so make sure the design is genuinely yours.